

Load data

On December 31, 2019, WHO was informed of several cases of pneumonia in Wuhan City, Hubei Province of China, originating from a new type of coronavirus. As the virus was unknown, the infection caused by it raised concerns because no one knew how it affected people.

To know the infection pathology and prognosis, daily level information on the affected people is essential. This can give some interesting insights when made available to medical professionals and researchers. A prolonged exploratory data analysis of this data can play an important role in drug discovery also.

For instance, the Johns Hopkins University has created an admirable portal using data on the affected cases extracted from the associated Google sheets. This data is available as .csv files in the Johns Hopkins University Center for Systems Science and Engineering (JHU CSSE) GitHub repository. From the URL, one can gather the most recent data on the worldwide spread of COVID-19. The file can be downloaded or read directly from https://github.com/CSSEGISandData/COVID-19/tree/master/csse_covid_19_data/csse_covid_19_time_series

```
#Data Acquisition
library(data.table)
df <- fread("https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_time_series/time_series_covid19_confirmed_global.csv")
df <- data.frame(df,
stringsAsFactors=FALSE)
```

Preprocessing

Preprocessing steps include filtering and data organisation. Filtering handles null value, missing and inappropriate value imputation, and data manipulation for the alignment of data with world map data.

Here I shall discuss *dplyr*, *ggplot2* and *ggmap* packages-based implementation of COVID-19 global

mapping. The COVID-19 confirmed records are loaded into the data frame *df*. As the function *data.frame()* with option *stringsAsFactors=FALSE* is used, all the variables are either characters or numeric, and conversion of factors to numeric and character is not required. However, it is required to replace all blank fields with appropriate character and numeric values. Since *na.omit()* removes the entire record containing NA, here this function is not used to manage null fields and instead, the individual fields are examined and replaced with appropriate Gauss values.

```
1. library(dplyr)
2. df <- transform(df, Province.State = ifelse(Province.State=="", Country.Region, Province.State))

#Replace all numeric fields with 0 if the value is NA
3. df <- df %>% replace(=="NA", 0) # replace with 0

4. countries <- df[,2:ncol(df)]
5. print(countries)
```

Global map marking

Getting a map of the world is very easy. In R, it is available within the *ggmap* package and can be called by the *map_data()* function. First load *ggmap* and then assign the world map to a local variable with:

```
library(ggmap)
map.world <- map_data("world")

head(map.world)
      long    lat group order region
subregion
1 -69.89912 12.45200  1  1 Aruba  <NA>
2 -69.89571 12.42300  1  2 Aruba  <NA>
3 -69.94219 12.43853  1  3 Aruba  <NA>
4 -70.00415 12.50049  1  4 Aruba  <NA>
5 -70.06612 12.54697  1  5 Aruba  <NA>
6 -70.05088 12.59707  1  6 Aruba  <NA>

> class(map.world)
[1] "data.frame"
```

To make the visualisation simple and decent, here we shall mark countries and localities (provinces) selectively. Within our dataframe *df*, the second column is the country (and region) variable and columns 5 onwards are date-wise infection counts. Therefore, columns two to *ncol(df)* are the required columns to mark the spread of COVID-19 within the world map.

```
countries <- df[,2:ncol(df)]
```

Since world-map marking requires the longitude and latitude, the third and fourth variables *Lat* and *Long* are two essential fields for this activity. One can inspect the values as shown below:

```
## INSPECT
as.factor(countries$Lat) %>% levels()
as.factor(countries$Long) %>% levels()
```

To map values, it is also essential to update country names to the matching country names of the world map. This can be done with the *recode()* function as shown in the following instruction:

```
# Recode names
countries$Country.Region <-
recode(countries$Country.Region
, 'United States' = 'US'
, 'United Kingdom' = 'UK'
))

head(countries[,c(1:3)])
  Country.Region    Lat
Long
1  Thailand  15.0000  101.0000
2   Japan  36.0000  138.0000
3  Singapore   1.2833  103.8333
4   Nepal  28.1667   84.2500
5  Malaysia   2.5000  112.5000
6   Canada  49.2827 -123.1207

class(countries)
[1] "data.frame"
```

For further processing, it is essential to convert country variables from